© Abdelouahab MOUSSAOUI



Ferhat Abbas Sétif 1 University
Faculty of Sciences
Department of Computer Science

CHAPTER 4

Recurrent Neural Networks (RNNs)

Prof. Abdelouahab Moussaoui

4th Year Engineering
Option: Artificial Intelligence
Academic Year: 2025-2026

"Recurrent networks: teaching machines to remember one time step at a time."

— Pr. Abdelouahab Moussaoui

Abstract

This chapter provides a comprehensive introduction to **Recurrent Neural Networks (RNNs)**, following established frameworks and definitions from leading educational resources. Starting from the limitations of feedforward networks for sequential data, we systematically explore the fundamental concepts: recurrent connections, hidden state, unfolding through time, and backpropagation through time (BPTT). We then dive into advanced architectures that address the vanishing gradient problem: Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU). Major sequence processing paradigms are covered, including many-to-one, one-to-many, many-to-many architectures, and the powerful encoder-decoder framework with attention mechanism. Practical implementation aspects are demonstrated through complete TensorFlow/Keras examples on sentiment analysis and language translation. By the end of this chapter, you will have a solid understanding of how RNNs work, how to design them, and how to apply them to real-world sequential data problems.

Contents

Preamble **5**

1 Introduction to Sequential Data **6**
1.1 Why Sequences Matter . . . 6
1.2 Limitations of Feedforward Networks for Sequences . . . 6
1.3 The Recurrent Idea . . . 6

2 Fundamental Principles of RNNs **7**
2.1 The Recurrent Connection . . . 7
2.1.1 Unfolding Through Time . . . 7
2.2 Hidden State: The Memory of RNNs . . . 8
2.3 Shared Weights Across Time . . . 9
2.4 Backpropagation Through Time (BPTT) . . . 9
2.5 Vanishing and Exploding Gradients . . . 10

3 RNN Architecture Variants by Input/Output Structure **10**
3.1 One-to-One (Vanilla) . . . 11
3.2 One-to-Many . . . 11
3.3 Many-to-One . . . 11
3.4 Many-to-Many (Synchronized) . . . 12
3.5 Many-to-Many (Asynchronous) . . . 12

4 Long Short-Term Memory (LSTM) **12**
4.1 LSTM Gates and Equations . . . 13
4.1.1 Forget Gate . . . 13
4.1.2 Input Gate . . . 13
4.1.3 Cell State Update . . . 13
4.1.4 Output Gate . . . 13
4.2 Advantages and Disadvantages of LSTM . . . 14

5 Gated Recurrent Units (GRU) **14**
5.1 GRU Equations . . . 15
5.1.1 Update Gate . . . 15
5.1.2 Reset Gate . . . 15
5.1.3 Candidate Hidden State . . . 15
5.1.4 Final Hidden State . . . 15
5.2 LSTM vs GRU Comparison . . . 16

6 Bidirectional RNNs **17**
6.1 Limitations of Bidirectional RNNs . . . 18

7 Encoder-Decoder Architecture and Attention **18**
7.1 The Encoder-Decoder Framework . . . 18
7.1.1 Encoder . . . 18
7.1.2 Decoder . . . 19
7.2 Attention Mechanism . . . 19
7.2.1 Attention Computation . . . 20
7.3 Advantages of Attention . . . 21

8 Major RNN Architectures and Variants **21**
8.1 Evolution of RNN Architectures . . . 21
8.2 Simple RNN (Elman Network) . . . 22
8.3 LSTM and GRU . . . 22
8.4 Bidirectional RNNs . . . 22
8.5 Stacked (Deep) RNNs . . . 22
8.6 The Transformer (Brief Introduction) . . . 23

9 Hands-On Example: RNN for Sentiment Analysis **25**
9.1 Dataset: IMDB Movie Reviews . . . 25
9.2 Model 1: Simple RNN Baseline . . . 25
9.3 Model 2: LSTM Model . . . 26
9.4 Model 3: Bidirectional LSTM . . . 26
9.5 Training the Models . . . 26
9.6 Evaluation and Comparison . . . 27
9.7 Making Predictions on New Reviews . . . 28

10 Advanced Example: Machine Translation with Attention **29**

11 Applications of Recurrent Neural Networks **32**
11.1 Natural Language Processing . . . 32
11.2 Time Series Forecasting . . . 32
11.3 Speech and Audio Processing . . . 33
11.4 Other Applications . . . 33

12 Challenges and Limitations of RNNs **34**
12.1 Training Difficulties . . . 34
12.2 Memory Limitations . . . 34
12.3 The Rise of Transformers . . . 34

Conclusion **34**

Preamble

About this Chapter

This chapter is part of the Deep Learning course for 4th year engineering students specializing in Artificial Intelligence. It focuses exclusively on Recurrent Neural Networks, building upon the foundational concepts covered in Chapter 2 (Deep Neural Networks) and Chapter 3 (Convolutional Neural Networks). The definitions and formulas presented draw upon widely adopted conventions from the deep learning literature [6].

Prerequisites

- Solid understanding of backpropagation, activation functions, and optimization (Chapter 2)
- Familiarity with Python and TensorFlow/Keras basics
- Basic understanding of sequences and time series data

Learning Objectives

- Understand the motivation and architecture behind RNNs for sequential data
- Master the mechanics of recurrent connections, hidden states, and unfolding through time
- Comprehend backpropagation through time (BPTT) and the vanishing gradient problem
- Analyze and compare LSTM and GRU architectures
- Understand different sequence processing paradigms (many-to-one, one-to-many, many-to-many)
- Implement RNNs using TensorFlow/Keras for text classification and generation tasks
- Apply advanced techniques like bidirectional RNNs and attention mechanisms

1 Introduction to Sequential Data

1.1 Why Sequences Matter

Many real-world problems involve data that is inherently sequential. Unlike static data (e.g., images), sequential data has a temporal or ordered structure where the order of elements carries crucial information.

Definition 1.1 : Sequential Data

Sequential data is a collection of observations arranged in a specific order, where the context and order of elements are essential for understanding. Each element depends on previous elements in the sequence.

Example 1.1 : Examples of Sequential Data

- **Time series**: stock prices, weather measurements, sensor readings
- **Natural language**: sentences, paragraphs, documents
- **Audio**: speech signals, music
- **Video**: sequences of frames
- **Biological data**: DNA sequences, protein structures

1.2 Limitations of Feedforward Networks for Sequences

Traditional feedforward neural networks (including CNNs) have fundamental limitations when processing sequential data:

Disadvantages

- ✗ **Fixed input size**: Feedforward networks require fixed-size inputs, but sequences have variable lengths
- ✗ **No memory**: Each input is processed independently without context from previous inputs
- ✗ **No parameter sharing**: Different positions in the sequence use different parameters, failing to capture position-invariant patterns
- ✗ **Order ignorance**: The network doesn't naturally leverage the sequential order of inputs

1.3 The Recurrent Idea

Recurrent Neural Networks (RNNs) address these limitations by introducing a critical innovation: **connections that loop back**, allowing information to persist across time steps.

Definition 1.2 : Recurrent Neural Network

A Recurrent Neural Network is a class of neural networks where connections between nodes form a directed graph along a temporal sequence. This allows the network to exhibit temporal dynamic behavior and maintain a **hidden state** that captures information from previous time steps.

The key insight is parameter sharing across time: the same weights are used at each time step, enabling the network to learn patterns regardless of their position in the sequence.

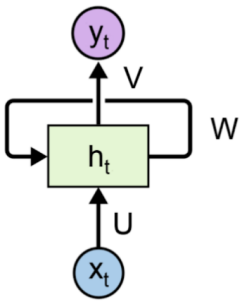


Figure 1: The core idea of RNNs: a loop that allows information to persist.

2 Fundamental Principles of RNNs

2.1 The Recurrent Connection

The defining characteristic of an RNN is the recurrent connection that feeds the hidden state from the previous time step back into the network.

2.1.1 Unfolding Through Time

To understand and train RNNs, we ”unfold” or ”unroll” the recurrent computation across time steps, creating a deep network where each layer corresponds to a time step but shares the same weights.

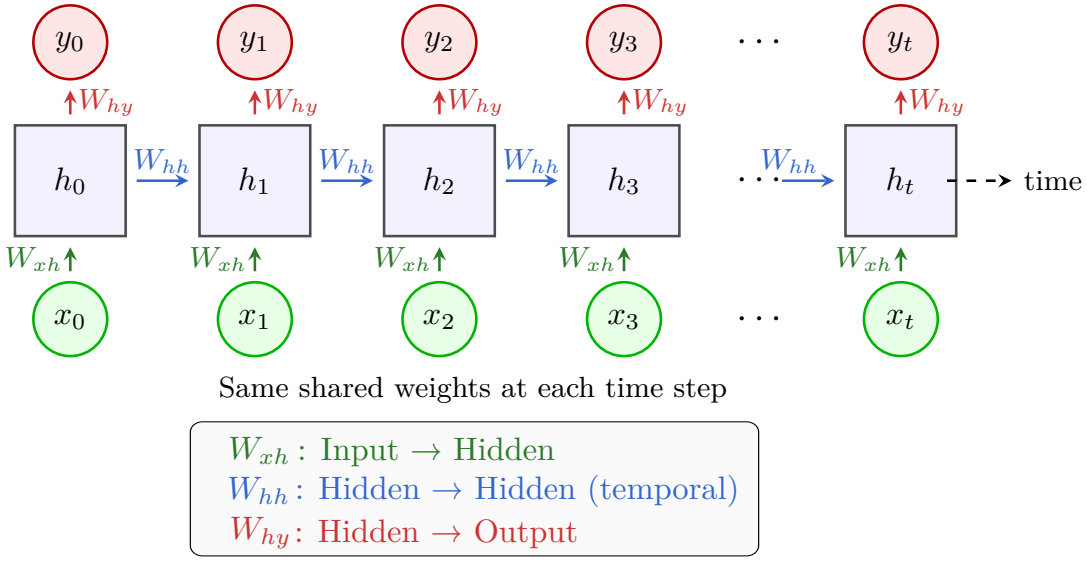


Figure 2: RNN folded (left) and unfolded through time (right). The same weights W, U, V are shared across all time steps.

RNN Forward Pass:

$$\mathbf{h}_t = f(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t)$$
$$\mathbf{y}_t = g(\mathbf{W}_{hy}\mathbf{h}_t)$$

Where:

- $\mathbf{x}_t$: input at time step t
- $\mathbf{h}_t$: hidden state at time t
- $\mathbf{y}_t$: output at time t
- $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$, $\mathbf{W}_{hy}$: weight matrices
- f: hidden layer activation (typically tanh or ReLU)
- g: output activation (softmax for classification, identity for regression)

2.2 Hidden State: The Memory of RNNs

The hidden state $\mathbf{h}_t$ serves as the network's memory, summarizing all relevant information from past inputs up to time t.

Definition 2.1 : Hidden State

The hidden state $\mathbf{h}_t$ is a vector that encapsulates the network's knowledge of the sequence up to time t. It is updated at each time step based on the current input and the previous hidden state.

2.3 Shared Weights Across Time

A fundamental property of RNNs is that the same weight matrices are used at every time step. This parameter sharing enables the network to:

Advantages

- Learn patterns regardless of their position in the sequence
- Process sequences of arbitrary length
- Dramatically reduce the number of parameters compared to using separate weights per time step

2.4 Backpropagation Through Time (BPTT)

Training RNNs requires a specialized version of backpropagation that accounts for the temporal dimension.

Definition 2.2 : Backpropagation Through Time

Backpropagation Through Time (BPTT) is the standard algorithm for training RNNs. It involves:

1. Unfolding the RNN through time for the entire sequence
2. Applying standard backpropagation on the unfolded network
3. Summing the gradients from all time steps for each shared weight

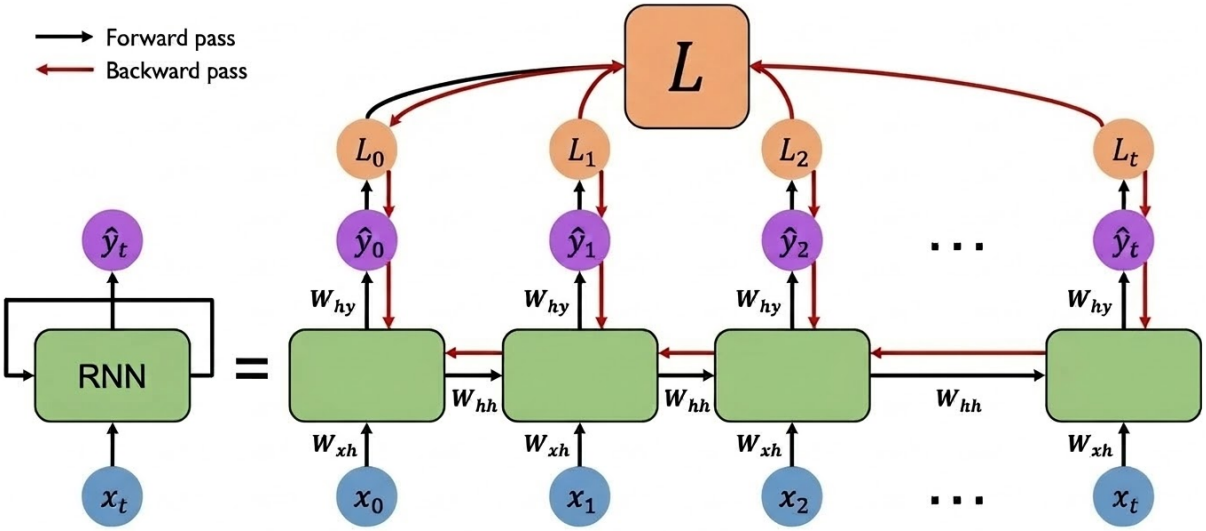


Figure 3: Backpropagation Through Time: gradients flow backward from loss at each time step through the unfolded network.

2.5 Vanishing and Exploding Gradients

While RNNs are powerful, they suffer from fundamental training difficulties when dealing with long sequences.

Theorem 2.1 : Vanishing Gradient Problem

During BPTT, gradients are multiplied by the same weight matrix $\mathbf{W}_{hh}$ at each time step. If the eigenvalues of $\mathbf{W}_{hh}$ are less than 1, gradients exponentially shrink to zero, preventing the network from learning long-range dependencies.

■ **Note** — The vanishing gradient problem is the primary motivation for more sophisticated RNN variants like LSTM and GRU, which incorporate gating mechanisms to control information flow.

Theorem 2.2 : Exploding Gradient Problem

Conversely, if eigenvalues are greater than 1, gradients exponentially grow, causing numerical instability and making training impossible.

3 RNN Architecture Variants by Input/Output Structure

According to the literature, RNNs can be configured with different input-output relationships. Four main patterns are identified:

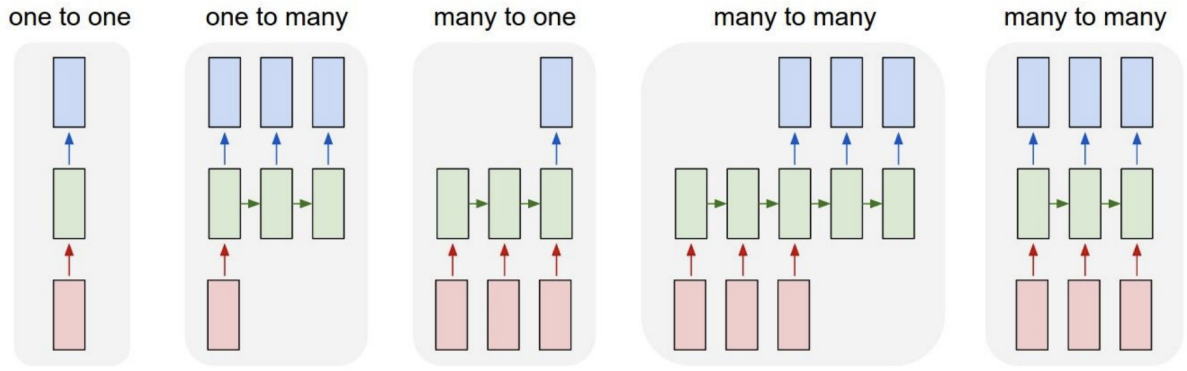


Figure 4: Different RNN architecture patterns for various tasks.

3.1 One-to-One (Vanilla)

Standard feedforward network without recurrence. Not typically considered an RNN architecture.

3.2 One-to-Many

Single input, sequence output.

- **Example**: Image captioning — one image input, sequence of words output
- **Architecture**: Input at first time step only, then generate sequence

Example 3.1 : Image Captioning

Given an image of a cat, the network generates the caption: "A cat sitting on a mat."

$$\textbf{Image} \rightarrow [\textbf{"A"}, \textbf{"cat"}, \textbf{"sitting"}, \textbf{"on"}, \textbf{"a"}, \textbf{"mat"}]$$

3.3 Many-to-One

Sequence input, single output.

- **Example**: Sentiment analysis — sequence of words, single sentiment class
- **Architecture**: Process entire sequence, use final hidden state for prediction

Example 3.2 : Sentiment Analysis

Given the movie review "This film was absolutely fantastic!", the network predicts: **Positive**

$$[\textbf{"This"}, \textbf{"film"}, \textbf{"was"}, \textbf{"absolutely"}, \textbf{"fantastic"}, \textbf{"!"}] \rightarrow \textbf{Positive}$$

3.4 Many-to-Many (Synchronized)

Sequence input, sequence output with aligned time steps.

- **Example**: Video frame classification — each frame labeled
- **Architecture**: Output at each time step

Example 3.3 : Video Frame Classification

For each frame in a video, classify the activity: running, jumping, sitting.

$$[\textbf{frame}_1, \textbf{frame}_2, \textbf{frame}_3, ...] \rightarrow [\textbf{"running"}, \textbf{"jumping"}, \textbf{"sitting"}, ...]$$

3.5 Many-to-Many (Asynchronous)

Sequence input, sequence output with possibly different lengths.

- **Example**: Machine translation — source language sequence to target language sequence
- **Architecture**: Encoder processes input sequence, decoder generates output sequence

Example 3.4 : Machine Translation

English sentence "Hello world" translated to French "Bonjour le monde".

$$[\textbf{"Hello"}, \textbf{"world"}] \rightarrow [\textbf{"Bonjour"}, \textbf{"le"}, \textbf{"monde"}]$$

4 Long Short-Term Memory (LSTM)

LSTM networks, introduced by Hochreiter & Schmidhuber in 1997, were designed specifically to overcome the vanishing gradient problem. They introduce a sophisticated gating mechanism that allows the network to learn what to remember, what to forget, and what to output.

Definition 4.1 : LSTM Cell

An LSTM cell maintains a **cell state** $\mathbf{c}_t$ (long-term memory) alongside the hidden state $\mathbf{h}_t$ (short-term memory). Three gates control information flow:

- **Forget gate**: what to discard from the cell state
- **Input gate**: what new information to store in the cell state
- **Output gate**: what to output based on the cell state

4.1 LSTM Gates and Equations

4.1.1 Forget Gate

Determines what information to discard from the cell state:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

4.1.2 Input Gate

Determines what new information to store:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

4.1.3 Cell State Update

Combines forgetting and new information:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

4.1.4 Output Gate

Determines what to output based on the cell state:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

LSTM Summary:

$$\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \\
\mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \\
\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \\
\mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \\
\mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t)
\end{aligned}$$

Where $\odot$ denotes element-wise multiplication.

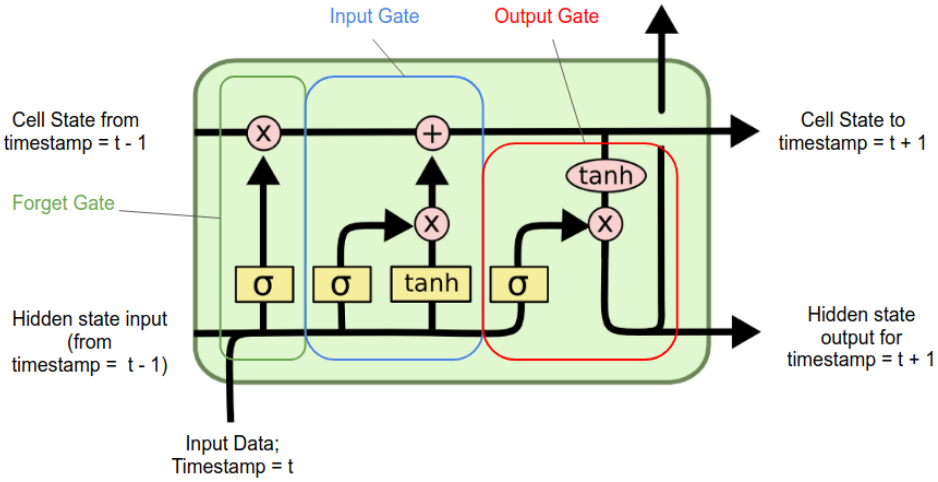


Figure 5: LSTM operations throw Forget, Input, update and Output Gates.

4.2 Advantages and Disadvantages of LSTM

Advantages

- ✓ **Long-range dependencies**: Can learn patterns spanning hundreds of time steps
- ✓ **Gated control**: Network learns what to remember and what to forget
- ✓ **Constant error flow**: Cell state provides a highway for gradients, mitigating vanishing gradients
- ✓ **State-of-the-art performance**: Excellent for many sequence tasks

Disadvantages

- ✗ **Computational cost**: More parameters and operations than simple RNN
- ✗ **Complexity**: Harder to understand and debug
- ✗ **Overfitting risk**: With many parameters, requires regularization
- ✗ **Training time**: Slower to train than simpler architectures

5 Gated Recurrent Units (GRU)

GRU, introduced by Cho et al. in 2014, is a simplified variant of LSTM that combines the forget and input gates into a single ”update gate” and merges the cell state with the hidden state.

Definition 5.1 : GRU Cell

GRU simplifies LSTM by using only two gates:

- **Update gate**: controls how much of the past information needs to be passed to the future
- **Reset gate**: controls how much of the past information to forget

5.1 GRU Equations

5.1.1 Update Gate

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z)$$

5.1.2 Reset Gate

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)$$

5.1.3 Candidate Hidden State

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \cdot [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h)$$

5.1.4 Final Hidden State

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

GRU Summary:

$$\begin{aligned}
\mathbf{z}_t &= \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \\
\mathbf{r}_t &= \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \\
\tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h) \\
\mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t
\end{aligned}$$

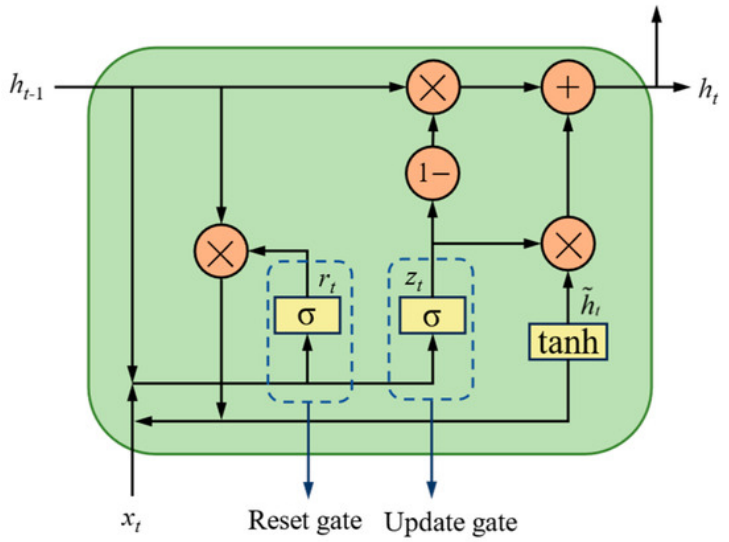


Figure 6: GRU cell architecture with update and reset gates.

5.2 LSTM vs GRU Comparison

Table 1: Comparison: LSTM vs GRU

Feature	**LSTM**	**GRU**
Gates	3 (forget, input, output)	2 (update, reset)
States	Cell state ($\mathbf{c}_t$) and hidden state ($\mathbf{h}_t$)	Only hidden state ($\mathbf{h}_t$)
Parameters	More (4 weight matrices)	Fewer (3 weight matrices)
Computation	More complex	Simpler, faster
Performance	Slightly better on very long sequences	Comparable, often similar
Typical use	When capturing very long dependencies	Good balance of performance and efficiency

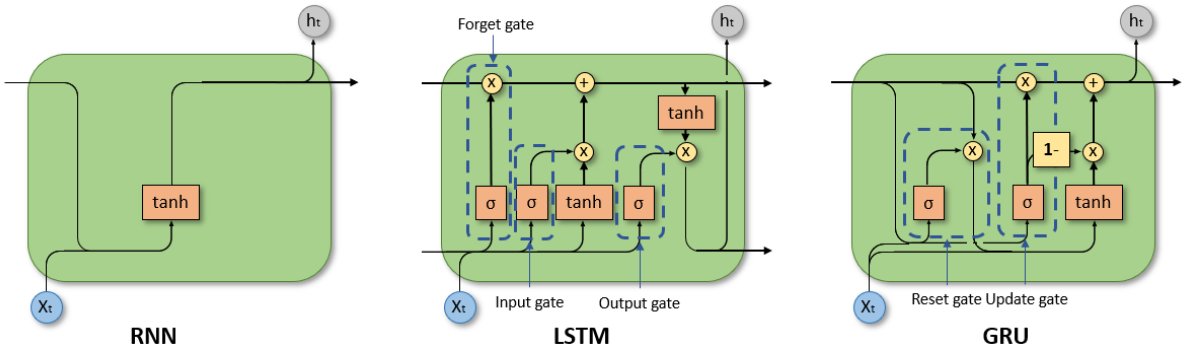


Figure 7: Architectural comparison between RNN (left), LSTM (center) and GRU (right).

■ **Note** — In practice, both LSTM and GRU perform similarly on many tasks. GRU is often preferred when computational resources are limited or when training smaller models, while LSTM might be chosen for tasks requiring very long-term dependencies.

6 Bidirectional RNNs

Standard RNNs process sequences in one direction (left to right), meaning the hidden state at time t only has information from past time steps. For many tasks, access to future context is equally important.

Definition 6.1 : Bidirectional RNN

A Bidirectional RNN processes the sequence in both forward and backward directions, concatenating the hidden states from both directions at each time step. This provides each time step with context from the entire sequence.

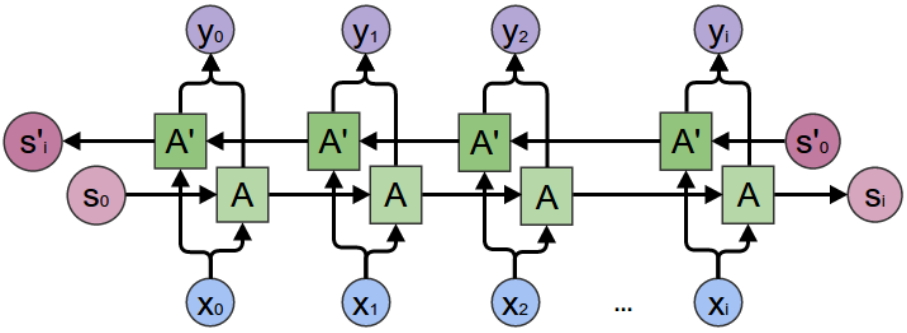


Figure 8: Bidirectional RNN architecture with forward and backward passes.

Bidirectional RNN:

$$\overrightarrow{\mathbf{h}}_t = \text{RNN}_{\text{forward}}(\mathbf{x}_t, \overrightarrow{\mathbf{h}}_{t-1})$$
$$\overleftarrow{\mathbf{h}}_t = \text{RNN}_{\text{backward}}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1})$$
$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \quad \text{(concatenation)}$$
$$\mathbf{y}_t = g(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y)$$

Example 6.1 : Applications of Bidirectional RNNs

- **Named Entity Recognition**: Identifying "New York" requires looking at both preceding and following words
- **Part-of-Speech Tagging**: The role of "record" depends on surrounding context
- **Speech Recognition**: Phoneme identification benefits from future audio context

6.1 Limitations of Bidirectional RNNs

Disadvantages

- ✗ Cannot be used for real-time applications where future context is unavailable (e.g., online speech recognition)
- ✗ Requires the entire sequence to be available before processing
- ✗ Doubles the parameter count and computational cost

7 Encoder-Decoder Architecture and Attention

7.1 The Encoder-Decoder Framework

For many sequence-to-sequence tasks (like machine translation), input and output sequences have different lengths. The encoder-decoder architecture addresses this challenge.

Definition 7.1 : Encoder-Decoder Architecture

The encoder-decoder framework consists of two RNNs:

- **Encoder**: Processes the input sequence and compresses it into a fixed-length context vector $\mathbf{c}$
- **Decoder**: Generates the output sequence conditioned on the context vector

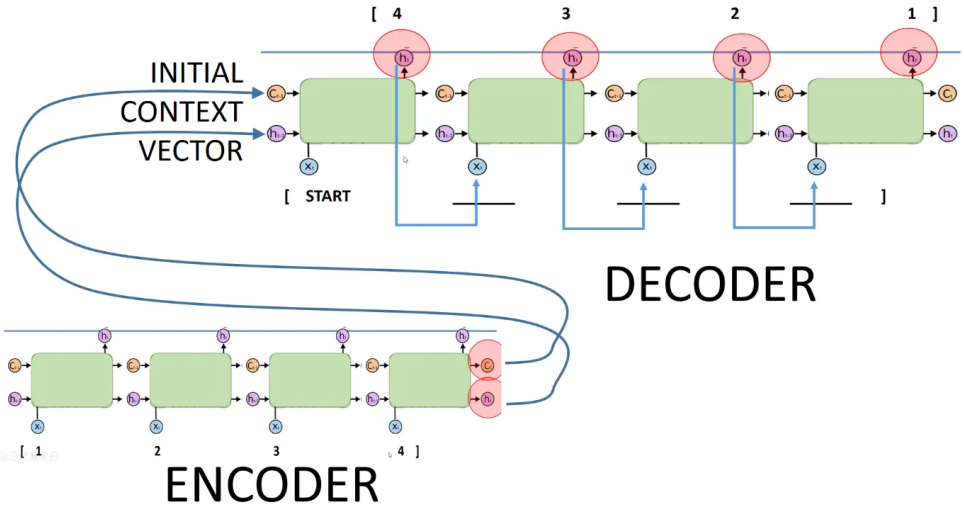


Figure 9: Encoder-decoder architecture for sequence-to-sequence tasks.

7.1.1 Encoder

The encoder reads the input sequence $\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_T$ and produces a sequence of hidden states $\mathbf{h}_1, \mathbf{h}_2, ..., \mathbf{h}_T$. The final hidden state (or a function of all hidden states) becomes

the context vector:

$$\mathbf{c} = q(\mathbf{h}_1, \mathbf{h}_2, ..., \mathbf{h}_T)$$

where q is typically the last hidden state $\mathbf{h}_T$ or an average of all states.

7.1.2 Decoder

The decoder generates the output sequence $\mathbf{y}_1, \mathbf{y}_2, ..., \mathbf{y}_{T'}$ conditioned on the context vector $\mathbf{c}$ and previously generated outputs:

$$p(\mathbf{y}_1, ..., \mathbf{y}_{T'}|\mathbf{x}_1, ..., \mathbf{x}_T) = \prod_{t=1}^{T'} p(\mathbf{y}_t|\mathbf{c}, \mathbf{y}_1, ..., \mathbf{y}_{t-1})$$



■ **Note** — The encoder-decoder framework with RNNs was a breakthrough for machine translation, but the fixed-length context vector becomes a bottleneck for long sequences.

7.2 Attention Mechanism

The attention mechanism, introduced by Bahdanau et al. in 2014, addresses the bottleneck problem by allowing the decoder to look at all encoder hidden states, not just a fixed context vector.

Definition 7.2 : Attention Mechanism

Attention allows the decoder to dynamically focus on different parts of the input sequence when generating each output word. At each decoding step, the model computes attention weights that determine which encoder hidden states are most relevant.

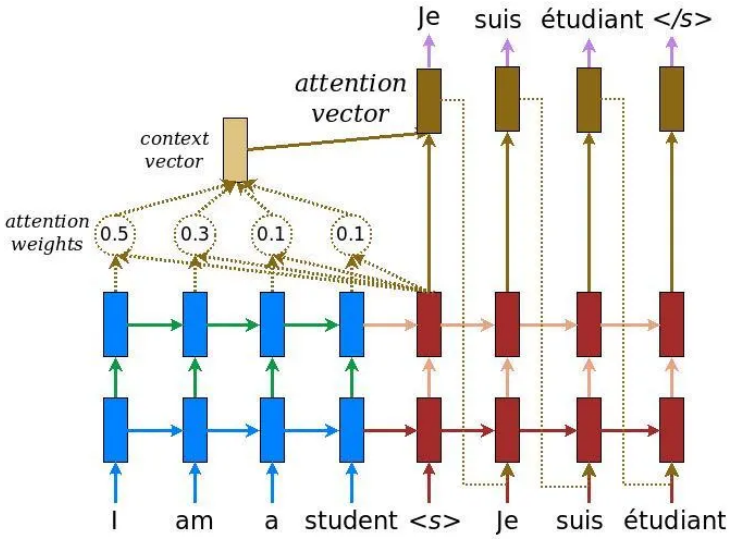


Figure 10: Attention mechanism: decoder focuses on relevant encoder states when generating each output.

7.2.1 Attention Computation

At each decoding step t:

1. Compute attention scores between decoder hidden state $\mathbf{s}_{t-1}$ and each encoder hidden state $\mathbf{h}_j$:
$$e_{tj} = \text{score}(\mathbf{s}_{t-1}, \mathbf{h}_j)$$
2. Convert scores to attention weights via softmax:
$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^{T} \exp(e_{tk})}$$
3. Compute context vector as weighted sum:
$$\mathbf{c}_t = \sum_{j=1}^{T} \alpha_{tj} \mathbf{h}_j$$
4. Combine context with decoder state to generate output:
$$\tilde{\mathbf{s}}_t = \tanh(\mathbf{W}_c[\mathbf{c}_t; \mathbf{s}_t])$$
$$p(\mathbf{y}_t | \mathbf{y}_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_s \tilde{\mathbf{s}}_t + \mathbf{b}_s)$$

Common Attention Scoring Functions:

$$\text{score}_{\text{dot}}(\mathbf{s}, \mathbf{h}) = \mathbf{s}^T \mathbf{h}$$
$$\text{score}_{\text{general}}(\mathbf{s}, \mathbf{h}) = \mathbf{s}^T \mathbf{W} \mathbf{h}$$
$$\text{score}_{\text{concat}}(\mathbf{s}, \mathbf{h}) = \mathbf{v}^T \tanh(\mathbf{W}[\mathbf{s}; \mathbf{h}])$$

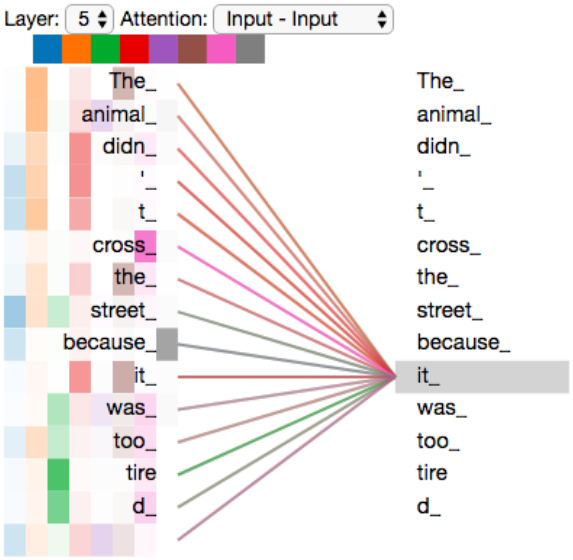


Figure 11: Attention weight visualization for machine translation. Darker squares indicate stronger alignment.

7.3 Advantages of Attention

Advantages

- ✓ **Handles long sequences**: No fixed-length bottleneck
- ✓ **Interpretability**: Attention weights show what the model focuses on
- ✓ **Improved performance**: Significant gains on translation, summarization, etc.
- ✓ **Gradient flow**: Direct connections between decoder and all encoder states

8 Major RNN Architectures and Variants

8.1 Evolution of RNN Architectures

Table 2: Major RNN Architectures - Categorized by Type

Year	Architecture	Key Innovation	Application Areas
1982	Hopfield Network	Recurrent, energy-based	Associative memory
1986	Simple RNN	Hidden state, BPTT	Early sequence modeling
1997	LSTM	Gating, cell state	Long-range dependencies
2000	GRU	Simplified gating	Efficient seq modeling
2005	Bidirectional RNN	Future context	NLP tasks
2014	Encoder-Decoder	Seq2Seq learning	Machine translation
2014	Attention Mechanism	Dynamic focusing	Translation, summary
2015	Neural Turing Machine	External memory	Algorithmic tasks
2017	Transformer	Self-attention	Modern NLP foundation

8.2 Simple RNN (Elman Network)

The original RNN formulation, still useful for understanding and simple tasks.

Advantages

- ✓ Simple to implement and understand
- ✓ Few parameters, fast training
- ✓ Works well for short sequences

Disadvantages

- ✗ Severe vanishing gradient problem
- ✗ Cannot capture long-range dependencies
- ✗ Rarely used in practice for real problems

8.3 LSTM and GRU

As covered in Sections 4 and 5, these are the workhorses of modern RNN applications.

8.4 Bidirectional RNNs

As covered in Section 6, these provide full sequence context.

8.5 Stacked (Deep) RNNs

Multiple RNN layers stacked on top of each other to learn hierarchical representations.

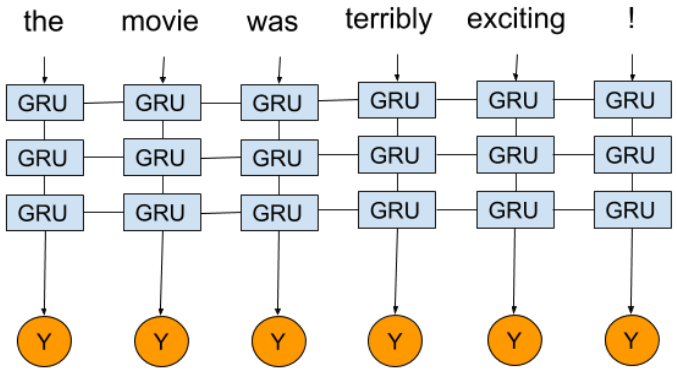


Figure 12: Stacked (deep) RNN architecture with multiple layers.

8.6 The Transformer (Brief Introduction)

While not an RNN, the Transformer architecture (Vaswani et al., 2017) has largely replaced RNNs in many applications.

Definition 8.1 : Transformer

The Transformer relies entirely on self-attention mechanisms, processing all positions in parallel rather than sequentially. This enables:

- Massive parallelization during training
- Direct connections between all positions
- Superior performance on long sequences

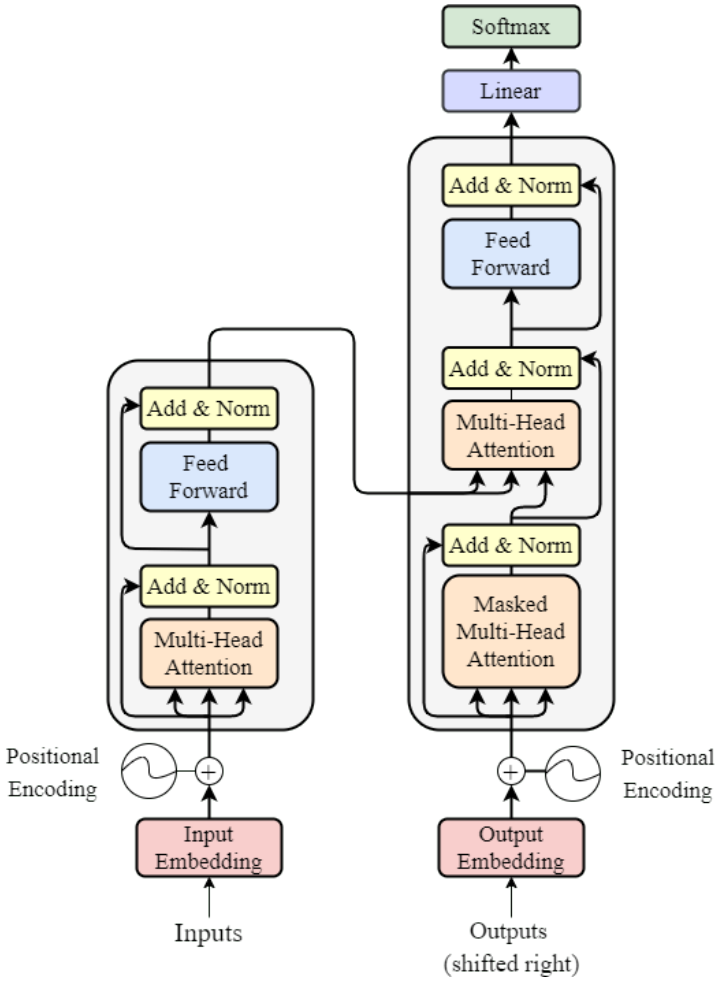


Figure 13: Simplified Transformer architecture (Vaswani et al., 2017).



■ **Note** — While Transformers have become dominant for NLP, RNNs (especially LSTM and GRU) remain relevant for:

- Streaming and real-time applications
- Resource-constrained environments
- Time series forecasting
- When sequence length is moderate

9 Hands-On Example: RNN for Sentiment Analysis

9.1 Dataset: IMDB Movie Reviews

The IMDB dataset contains 50,000 movie reviews labeled as positive or negative.

```
import tensorflow as tf
from tensorflow.keras import layers, models, datasets
from tensorflow.keras.preprocessing import sequence
import numpy as np
import matplotlib.pyplot as plt

# Load IMDB dataset (words indexed by frequency)
max_features = 20000  # Vocabulary size
maxlen = 500  # Maximum sequence length

(x_train, y_train), (x_test, y_test) = datasets.imdb.load_data(
    num_words=max_features
)

# Pad sequences to fixed length
x_train = sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = sequence.pad_sequences(x_test, maxlen=maxlen)

print(f"Training data shape: {x_train.shape}")
print(f"Test data shape: {x_test.shape}")
```

9.2 Model 1: Simple RNN Baseline

```
def create_simple_rnn_model():
    model = models.Sequential([
        layers.Embedding(max_features, 128, input_length=maxlen),
        layers.SimpleRNN(64, dropout=0.2, return_sequences=False),
        layers.Dense(64, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(1, activation='sigmoid')
    ])
    return model

simple_rnn = create_simple_rnn_model()
simple_rnn.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
simple_rnn.summary()
```

9.3 Model 2: LSTM Model

```
def create_lstm_model():
    model = models.Sequential([
        layers.Embedding(max_features, 128, input_length=maxlen),
        layers.LSTM(64, dropout=0.2, recurrent_dropout=0.2,
           return_sequences=False),
        layers.Dense(64, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(1, activation='sigmoid')
    ])
    return model

lstm_model = create_lstm_model()
lstm_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
lstm_model.summary()
```

9.4 Model 3: Bidirectional LSTM

```
def create_bidirectional_lstm_model():
    model = models.Sequential([
        layers.Embedding(max_features, 128, input_length=maxlen),
        layers.Bidirectional(layers.LSTM(64, dropout=0.2,
           recurrent_dropout=0.2)),
        layers.Dense(64, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(1, activation='sigmoid')
    ])
    return model

bidirectional_model = create_bidirectional_lstm_model()
bidirectional_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
bidirectional_model.summary()
```

9.5 Training the Models

```
# Use a smaller subset for faster demonstration
x_train_small = x_train[:10000]
y_train_small = y_train[:10000]
```

```
x_val = x_train[10000:15000]
y_val = y_train[10000:15000]

# Callbacks
callbacks = [
    tf.keras.callbacks.EarlyStopping(patience=3, restore_best_weights
        =True),
    tf.keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=2)
]

# Train Simple RNN
print("Training Simple RNN...")
history_simple = simple_rnn.fit(
    x_train_small, y_train_small,
    batch_size=64,
    epochs=10,
    validation_data=(x_val, y_val),
    callbacks=callbacks,
    verbose=1
)

# Train LSTM
print("\nTraining LSTM...")
history_lstm = lstm_model.fit(
    x_train_small, y_train_small,
    batch_size=64,
    epochs=10,
    validation_data=(x_val, y_val),
    callbacks=callbacks,
    verbose=1
)

# Train Bidirectional LSTM
print("\nTraining Bidirectional LSTM...")
history_bidirectional = bidirectional_model.fit(
    x_train_small, y_train_small,
    batch_size=64,
    epochs=10,
    validation_data=(x_val, y_val),
    callbacks=callbacks,
    verbose=1
)
```

9.6 Evaluation and Comparison

```
# Evaluate on test set
test_loss_simple, test_acc_simple = simple_rnn.evaluate(x_test,
    y_test)
```

```
test_loss_lstm, test_acc_lstm = lstm_model.evaluate(x_test, y_test)
test_loss_bi, test_acc_bi = bidirectional_model.evaluate(x_test,
    y_test)

print(f"Simple RNN Test Accuracy: {test_acc_simple:.4f}")
print(f"LSTM Test Accuracy: {test_acc_lstm:.4f}")
print(f"Bidirectional LSTM Test Accuracy: {test_acc_bi:.4f}")

# Plot training history
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history_simple.history['accuracy'], label='Simple RNN')
plt.plot(history_lstm.history['accuracy'], label='LSTM')
plt.plot(history_bidirectional.history['accuracy'], label='BiLSTM')
plt.title('Training Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history_simple.history['loss'], label='Simple RNN')
plt.plot(history_lstm.history['loss'], label='LSTM')
plt.plot(history_bidirectional.history['loss'], label='BiLSTM')
plt.title('Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.savefig('rnn_comparison.png')
plt.show()
```

9.7 Making Predictions on New Reviews

```
# Function to decode indices back to words
word_index = datasets.imdb.get_word_index()
reverse_word_index = {value: key for key, value in word_index.items()
    }

def decode_review(encoded_review):
    return ' '.join([reverse_word_index.get(i - 3, '?') for i in
        encoded_review if i > 0])

# Function to predict sentiment
def predict_sentiment(model, text, tokenizer, maxlen=500):
    # This is simplified - in practice, you'd need to properly
        tokenize new text
```

```
    # For demonstration, we'll use a pre-encoded sample
    sequence = tokenizer.texts_to_sequences([text])
    padded = sequence.pad_sequences(sequence, maxlen=maxlen)
    prediction = model.predict(padded)[0][0]
    sentiment = "Positive" if prediction > 0.5 else "Negative"
    confidence = prediction if prediction > 0.5 else 1 - prediction
    return sentiment, confidence

# Example (requires proper preprocessing in practice)
# sentiment, confidence = predict_sentiment(lstm_model, "This movie was fantastic!", some_tokenizer)
# print(f"Sentiment: {sentiment}, Confidence: {confidence:.4f}")
```

10 Advanced Example: Machine Translation with Attention

This section demonstrates a more complex encoder-decoder architecture with attention for machine translation.

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np

# Simplified example structure - full implementation would require
# proper data preprocessing with tokenization

class Encoder(layers.Layer):
    def __init__(self, vocab_size, embedding_dim, enc_units, batch_sz):
        super(Encoder, self).__init__()
        self.batch_sz = batch_sz
        self.enc_units = enc_units
        self.embedding = layers.Embedding(vocab_size, embedding_dim)
        self.gru = layers.GRU(
            self.enc_units,
            return_sequences=True,
            return_state=True,
            recurrent_initializer='glorot_uniform'
        )

    def call(self, x, hidden):
        x = self.embedding(x)
        output, state = self.gru(x, initial_state=hidden)
        return output, state

    def initialize_hidden_state(self):
        return tf.zeros((self.batch_sz, self.enc_units))

```

```
class BahdanauAttention(layers.Layer):
    def __init__(self, units):
        super(BahdanauAttention, self).__init__()
        self.W1 = layers.Dense(units)
        self.W2 = layers.Dense(units)
        self.V = layers.Dense(1)

    def call(self, query, values):
        # query: hidden state shape (batch_size, hidden_size)
        # values: encoder outputs shape (batch_size, max_length,
           hidden_size)

        # Expand query to broadcast
        query_with_time_axis = tf.expand_dims(query, 1)

        # Calculate attention scores
        score = self.V(tf.nn.tanh(
            self.W1(query_with_time_axis) + self.W2(values)
        ))

        # Attention weights
        attention_weights = tf.nn.softmax(score, axis=1)

        # Context vector
        context_vector = attention_weights * values
        context_vector = tf.reduce_sum(context_vector, axis=1)

        return context_vector, attention_weights

class Decoder(layers.Layer):
    def __init__(self, vocab_size, embedding_dim, dec_units, batch_sz
       ):
        super(Decoder, self).__init__()
        self.batch_sz = batch_sz
        self.dec_units = dec_units
        self.embedding = layers.Embedding(vocab_size, embedding_dim)
        self.gru = layers.GRU(
            self.dec_units,
            return_sequences=True,
            return_state=True,
            recurrent_initializer='glorot_uniform'
        )
        self.fc = layers.Dense(vocab_size)
        self.attention = BahdanauAttention(self.dec_units)

    def call(self, x, hidden, enc_output):
        # x shape: (batch_size, 1)
        # hidden shape: (batch_size, hidden_size)
        # enc_output shape: (batch_size, max_length, hidden_size)
```

```
76
77          context_vector, attention_weights = self.attention(hidden,
                enc_output)
78
79          # Embedding
80          x = self.embedding(x)
81
82          # Concatenate context vector with embedded input
83          x = tf.concat([tf.expand_dims(context_vector, 1), x], axis
                =-1)
84
85          # Pass through GRU
86          output, state = self.gru(x)
87
88          # Reshape for dense layer
89          output = tf.reshape(output, (-1, output.shape[2]))
90
91          # Final output
92          x = self.fc(output)
93
94          return x, state, attention_weights
95
96  # Full translation model would integrate these components
97  # with training loop and inference logic
```

11 Applications of Recurrent Neural Networks

11.1 Natural Language Processing

Table 3: RNN Applications in NLP

Application	Architecture	Description
Sentiment Analysis	Many-to-One	Classify text sentiment (positive/negative)
Machine Translation	Encoder-Decoder + Attention	Translate between languages
Text Generation	One-to-Many	Generate new text character by character
Named Entity Recognition	Many-to-Many (sync)	Tag entities (person, location, organization)
Part-of-Speech Tagging	Many-to-Many (sync)	Assign grammatical tags
Question Answering	Encoder-Decoder	Generate answers to questions
Summarization	Encoder-Decoder	Create concise summaries

11.2 Time Series Forecasting

Table 4: RNN Applications in Time Series

Application	Architecture	Description
Stock Price Prediction	Many-to-One	Forecast future prices
Weather Forecasting	Many-to-Many	Predict temperature, precipitation
Energy Load Forecasting	Many-to-One	Predict electricity demand
Anomaly Detection	Many-to-One	Identify unusual patterns
Sensor Data Analysis	Many-to-Many	Process IoT sensor streams

11.3 Speech and Audio Processing

Table 5: RNN Applications in Speech/Audio

Application	Architecture	Description
Speech Recognition	Many-to-Many	Convert speech to text
Music Generation	One-to-Many	Generate musical sequences
Speaker Identification	Many-to-One	Identify who is speaking
Emotion Recognition	Many-to-One	Detect emotion from voice

11.4 Other Applications

- **Video Analysis**: Action recognition, video captioning
- **Bioinformatics**: Protein structure prediction, DNA sequence analysis
- **Handwriting Recognition**: Convert handwritten text to digital text
- **Robotics**: Motion planning, control sequences
- **Recommendation Systems**: Sequential recommendation based on user history

12 Challenges and Limitations of RNNs

12.1 Training Difficulties

- **Vanishing/Exploding Gradients**: Despite LSTM/GRU, very long sequences remain challenging
- **Sequential Training**: Cannot parallelize across time steps, leading to slow training
- **Long Training Times**: Especially for long sequences and large datasets

12.2 Memory Limitations

- **Finite Context**: Even LSTM has limited practical memory (typically < 1000 steps)
- **Forgetting**: Tendency to forget early parts of very long sequences

12.3 The Rise of Transformers

Transformers address many RNN limitations:

Advantages

✓ **Parallelization**: Process all positions simultaneously

✓ **Long-range dependencies**: Direct connections between all positions

✓ **Better performance**: Superior results on most NLP benchmarks



■ **Note** — Despite Transformers' dominance, RNNs remain relevant for:

- Streaming applications requiring low latency
- Resource-constrained environments
- Time series forecasting
- Understanding recurrent concepts before moving to Transformers

Conclusion

Recurrent Neural Networks represent a fundamental shift in neural network design, enabling the processing of sequential data with variable length and capturing temporal dependencies. From the simple RNN through LSTM and GRU to attention mechanisms, these architectures have revolutionized natural language processing, time series analysis, and speech recognition.

Key Takeaways

- RNNs maintain a hidden state that acts as memory, allowing information to persist across time steps
- Weight sharing across time enables processing of variable-length sequences
- Backpropagation Through Time (BPTT) is the training algorithm, but suffers from vanishing gradients
- LSTM and GRU use gating mechanisms to mitigate vanishing gradients and capture long-range dependencies
- Different architectural patterns (many-to-one, one-to-many, many-to-many) address different task types
- Bidirectional RNNs provide future context when available
- Encoder-decoder with attention enables sequence-to-sequence tasks like translation
- While Transformers have become dominant for NLP, RNNs remain relevant for specific applications

Looking Forward

In the next chapter, we will explore **Attention Mechanisms and Transformers** in depth, building on the attention concepts introduced here. We'll see how self-attention and parallel processing have revolutionized deep learning for sequences, and understand why Transformers have become the foundation of modern large language models.

> *"Memory is the bridge between past and future —*
> *and RNNs teach machines how to build that bridge."*

References

[1] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.

[2] Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*.

[3] Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.

[4] Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.

[5] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

[6] Stanford CS 230 Deep Learning Cheatsheets. (2018). Recurrent Neural Networks cheatsheet.

[7] Graves, A. (2012). Supervised sequence labelling with recurrent neural networks. *Studies in Computational Intelligence*.